

# Report Mate — Technical Document

Show Me Your Agents Hackathon (NUS-ISS, Public Category)

Team Simpsons — Arrav · Aaron · Ngiam · Letao

2026

## Contents

|                                                 |          |
|-------------------------------------------------|----------|
| <b>1. Overview</b>                              | <b>2</b> |
| 1.1 Design principles . . . . .                 | 2        |
| <b>2. System architecture</b>                   | <b>2</b> |
| 2.1 Backend package layout . . . . .            | 2        |
| 2.2 Data model . . . . .                        | 3        |
| 2.3 Database migrations . . . . .               | 3        |
| <b>3. The AI agent</b>                          | <b>3</b> |
| 3.1 Manual tool-calling loop . . . . .          | 3        |
| 3.2 Agent tools . . . . .                       | 4        |
| 3.3 One-shot and conversational modes . . . . . | 4        |
| 3.4 Safety and correctness properties . . . . . | 4        |
| <b>4. Security</b>                              | <b>4</b> |
| <b>5. Deployment</b>                            | <b>5</b> |
| <b>6. Testing and verification</b>              | <b>5</b> |
| <b>7. Technology choices and constraints</b>    | <b>5</b> |

# 1. Overview

Report Mate is a mobile-first web application that helps field-service technicians write customer visit reports. A dispatcher/admin builds report **templates** from typed field blocks; a technician fills a chosen template either by hand or by giving an **AI agent** a rough, free-text (or dictated) account of the visit. The agent fills the template’s blanks, pulls in supporting context (customer job history and a parts catalog), and flags any required field it cannot confidently complete. The technician always reviews and edits the draft before submitting — the agent drafts, the human stays the author of record.

The application was built for the “Show Me Your Agents” hackathon, which is judged partly on agentic AI use (including a “Best Agents Use” award). The AI agent is therefore a first-class capability, not an add-on.

## 1.1 Design principles

- **Human-in-the-loop always.** No report reaches “submitted” without a human review step. The agent leaves report status as **draft**.
- **Graceful degradation.** If the AI gateway is unavailable, the same template can still be filled by hand. The product does not hard-depend on the AI.
- **Predictable over clever.** The agent fills schema-defined fields and flags uncertainty; it never improvises content outside the template’s structure.
- **Mobile-first, field conditions.** Large tap targets, one-handed use, high contrast for outdoor readability.

# 2. System architecture

Report Mate is a two-tier application: a React + TypeScript single-page app and a Go REST/JSON backend, backed by PostgreSQL. In production a single Go binary serves both the JSON API and the built frontend on one origin, which keeps the session cookie same-origin.

| Layer      | Technology                                                   |
|------------|--------------------------------------------------------------|
| Frontend   | React + TypeScript (Vite), mobile-first                      |
| Backend    | Go, REST/JSON API                                            |
| Database   | PostgreSQL (AWS Lightsail-managed in production)             |
| Hosting    | AWS Lightsail instance (Singapore region)                    |
| AI gateway | Organizer-provided, Bedrock-backed, OpenAI/Ollama-compatible |

## 2.1 Backend package layout

The backend follows one Go package per domain under `internal/`, avoiding a single flat API package:

- **auth** — cookie-based session authentication, registration, session lifecycle.
- **accounts / accountapi** — company accounts, admin/worker authorization codes, worker and profile management.

- **middleware** — identity, role-based access control (RBAC), request logging, panic recovery.
- **templates** — report template CRUD and schema validation.
- **reports** — service reports, content validation, HTML/PDF export.
- **jobs** — job records and customer history provider.
- **parts** — parts catalog provider.
- **agent** — the LLM gateway client and the tool-calling loop (the only package that talks to the gateway).
- **dashboard** — dispatcher job/report history and CSV export.
- **config, db, httpx, security** — configuration, migrations, HTTP helpers, and secret handling.

## 2.2 Data model

- **users** — id, name, role
- **jobs** — id, customer, address, scheduled\_at, status
- **report\_templates** — id, name, schema (JSONB: sections → fields → type / required)
- **service\_reports** — id, job\_id, technician\_id, template\_id, content (JSONB), filled\_by (agent | manual | mixed), status, timestamps
- **parts\_used** — report\_id, part, qty
- **parts\_catalog** — the master list of parts the agent matches free-text mentions against
- **attachments** — report\_id, storage\_key, type
- **Accounts/auth tables** — companies, sessions, authorization codes, login identities

The template schema JSON stored in `report_templates.schema` is the single contract shared by the template builder, the report renderer, and the agent's `get_template_schema` tool.

## 2.3 Database migrations

Schema is applied through ordered, additive SQL migrations embedded in the Go binary and run automatically at startup. The migration runner records each applied file by full filename in a `schema_migrations` table, inside the same transaction as the migration itself, so schema and its record commit or roll back together. Development-only seed data (dev users, demo jobs) is excluded from the production migration stream and applied separately by an explicit `cmd/devseed` command.

# 3. The AI agent

The agent is the product's headline capability and its most deliberate piece of engineering.

## 3.1 Manual tool-calling loop

The agent is a **hand-rolled JSON tool-calling loop** in `internal/agent`. It does **not** use the gateway's native tool-calling (which has documented reliability problems on the current build), and it does **not** introduce LangChain, LangGraph, or a second runtime. The loop:

1. Sends a deterministic system prompt (the agent's job, the tool contract, the template schema, and the current field values) plus the conversation transcript.
2. Receives a reply constrained to a single JSON object, e.g. `{"tool": "fill_field", "field_id": "notes", "value": "the filter."}`.
3. Parses that instruction (stripping any Markdown code fence), dispatches to the real Go function, and feeds the result back as the next message.

4. Repeats until the model emits `save_draft`, asks the technician a question, or a budget is reached.

A subtle but critical gateway detail: because the proxy is Bedrock-backed and rejects native tool-result roles, tool results are fed back as ordinary user messages prefixed with `TOOL RESULT:` rather than as a `role: "tool"` message.

### 3.2 Agent tools

- `get_template_schema` — which fields exist and which are required.
- `get_job_history` — the customer's past jobs, for context.
- `get_parts_catalog` — match mentioned parts to real catalog entries.
- `fill_field` — write one field of the draft.
- `flag_missing_field` — mark a required field it could not confidently fill.
- `ask_technician` — ask one clarifying question and pause the turn (conversational mode).
- `save_draft` — persist the draft for human review and finish the run.

### 3.3 One-shot and conversational modes

The agent supports two modes over the same runner:

- **One-shot fill** (POST `/api/reports/{id}/agent-fill`): the technician provides a single free-text account; the agent fills what it can in one pass.
- **Conversational fill** (POST `/api/reports/{id}/agent-chat`): the agent may ask clarifying questions across multiple turns, carrying the transcript on the client. Turn and question budgets (defaults: 6 turns, 4 questions) bound how much a single conversation can spend against the shared credit pool.

### 3.4 Safety and correctness properties

- **Field-scope security.** `save_draft` persists through the existing `reports.ValidateContent` boundary, which rejects any write to a field id not declared by the template schema snapshot. The agent has no route to arbitrary data changes.
- **Atomicity.** The runner mutates only an in-memory copy of the report content and persists once, on `save_draft`. A run that fails before then leaves the stored draft untouched.
- **Bounded runs.** An overall time budget and an iteration cap guarantee the loop terminates.
- **Key confinement.** The gateway API key lives only in backend configuration. It is never sent to the frontend and never logged.
- **Cost logging.** Every call records the returned token usage so credit spend can be tracked.

These properties are covered by a suite of property-based and integration tests in `internal/agent`.

## 4. Security

- **Authentication** uses opaque, database-backed session cookies (HttpOnly; Secure in production). Every authenticated request loads the current principal, so deactivation and role changes take effect immediately.
- **Authorization** is enforced by RBAC middleware on privileged routes (e.g. dispatcher-only template management). A technician can only reach their own reports.

- **Registration** requires a company authorization code, so account type cannot be self-granted from the browser.
- **Secret handling.** The database URL and the LLM gateway key are held only in backend configuration, never logged, never returned to the client. Passwords are stored as bcrypt hashes; authorization codes are stored as SHA-256 hashes.

## 5. Deployment

The application deploys as a single statically linked Go binary plus the built frontend, on an AWS Lightsail instance in the Singapore region, backed by a Lightsail-managed PostgreSQL database.

- The binary serves the API and the SPA on one origin. Non-API routes fall back to `index.html` so client-side deep links and refreshes work; API routes are matched first and never masked by the SPA.
- Configuration is supplied through environment variables (database URL, session signing key, gateway URL/key/model, static directory, export directory).
- Migrations run automatically on boot; the server refuses to start without a database URL.
- The process runs under systemd, so it restarts on failure and survives instance reboots.

A repeatable `scripts/build-release.sh` builds the frontend and backend into a `release/` bundle ready to copy to the instance, and `docs/deployment.md` documents the full provisioning and run procedure.

## 6. Testing and verification

The backend carries unit, property-based, and integration tests across the auth, accounts, reports, dashboard, config, database, and agent packages. Pure-logic tests run without a database; database-backed tests skip cleanly when no test database is configured. The agent suite verifies the tool-calling loop's correctness properties (field-scope security, atomicity, termination, key confinement) directly. The frontend is type-checked and unit-tested, and the production build is verified before release.

## 7. Technology choices and constraints

- **No LangChain/LangGraph** for the agent loop — the gateway does not fully support it; a hand-rolled Go tool-calling loop is used instead.
- **No native gateway tool-calling** — documented reliability problems; the manual JSON pattern is used and was verified against the live gateway.
- **PostgreSQL on AWS Lightsail** — the hackathon sandbox AWS account is scoped to Lightsail (not RDS/Aurora/general EC2).
- **The gateway is called only from the Go backend** — never from the frontend, so the shared API key is never exposed.